Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO2 – Debugging and Optimisation Task
Course Outcome Assessed	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems.
Maximum Marks	**50**
Duration	1.5 hrs (Assessment conducted in class)
Mode	Individual
Student Reg. No.	192525477
Student Name	SWATHI K

Instructions: Debug the code by reasoning and testing, not by rewriting it from scratch. For every change, state what was wrong, why it was wrong and how you verified the fix. Optimisations must preserve the corrected outputs. Marks for each part are shown in brackets.

DEBUGGING AND OPTIMISATION TASK (50 MARKS)

Scenario – SOC Alert-Triage Module of "SentinelOps": SentinelOps is the Security Operations Centre (SOC) of a cloud-services company. A junior developer wrote the Python module below to (1) sort incoming alerts by severity, highest first, keeping alerts of equal severity in arrival order; (2) check whether a source IP address is on the blocklist; (3) find every position of an attack signature in a log line; and (4) find the two login attempts whose locations are closest to each other, which the SOC uses to flag clustered credential-stuffing attacks. The module passed code review but failed in production: alerts disappeared or were duplicated, blocked IP addresses got through, signatures at the end of a log were missed while false signatures were reported, and the "closest logins" report flagged logins hundreds of kilometres apart. The module must be fixed before it processes about 50,000 alerts and batches of 8,000 login events per day.

Code under investigation:

```
import math

def sort_alerts(alerts):            # insertion sort by severity, highest first
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j > 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j] = key
    return alerts

def is_blocked(blocklist, ip):      # sequential search
    for i in range(len(blocklist)):
        if blocklist[i] == ip:
            return False
        else:
            return False

def find_signature(log, pattern):   # brute-force string matching, all occurrences
    n, m = len(log), len(pattern)
    hits = []
    for i in range(n - m):
        for j in range(m):
            if log[i + j] != pattern[j]:
                break
        if j == m - 1:
            hits.append(i)
    return hits

def closest_logins(events):         # brute-force closest pair of login locations
    min_dist = 0
    pair = None
    for i in range(len(events)):
        for j in range(len(events)):
            d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
                          (events[i][1] - events[j][1]) ** 2)
            if d > min_dist:
                min_dist = d
                pair = (i, j)
    return pair, min_dist
```

Test data:

```
alerts    = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
queries   = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
log, pattern = "FAILFAIXFAIL", "FAIL"
events    = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]   # (x, y) in
km
```

(a) Reproduce and Identify the Defects (10 Marks): Run the module on the test data. For each function, record the expected output and the actual output, then locate every defect: state the function, the faulty line and the nature of the error. The module contains nine defects, and none of them is marked in the code.

(b) Explain the Root Causes and Debug Systematically (12 Marks): For each defect, explain its root cause and how it produces the observed wrong output. Correct the functions one at a time, supporting your reasoning with a step-by-step trace (for example, the insertion trace for the alerts list and the alignment table for the log), and re-test after every correction. Include boundary tests such as an empty blocklist, a pattern at the very end of the log, a pattern longer than the log, equal severities and fewer than two login events.

(c) Optimise the Corrected Module (10 Marks): Without changing any output, improve the corrected module for the production workload. Justify whether insertion sort remains suitable for nearly sorted alerts; propose a better approach for repeated blocklist lookups; make sure the signature scan performs no redundant comparisons; and, for closest_logins, eliminate self and

repeated pairs, remove unnecessary square-root computations and prune pairs that cannot be closest.

(d) Analyse Before and After (10 Marks): Derive the number of basic operations and the time complexity of each function before and after optimisation. Measure the distance computations of the brute-force and optimised closest_logins for at least three batch sizes between 1,000 and 8,000 events, and compare insertion-sort comparisons on nearly sorted and random alerts. Explain whether the measurements agree with your analysis.

(e) Deliver Production-Quality Code (8 Marks): Submit the final, optimised module with docstrings, meaningful comments and a test harness that checks every function, including the boundary cases from (b). Commit the code to your GitHub repository and share the link through Google Classroom.

Rubrics – Debugging and Optimisation Task

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Code of Conduct

I SWATHI K (192525477) (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: SWATHI K Date: 26/10/2026

Purpose and Scope

This report investigates four algorithms used in a security alert-triage module: insertion sort, sequential blocklist search, brute-force pattern matching, and closest-pair search. It first establishes the defects in the supplied implementation, then explains and tests corrections, proposes optimisations, analyses complexity, and provides a final Python module with a test harness. The report distinguishes the supplied code's actual behaviour from the corrected expected behaviour.

Instructions and Academic Integrity

The task requires debugging by reasoning and testing rather than replacing the code without explanation. For every correction, document what was wrong, why it was wrong, and how the fix was verified. Optimisation must preserve the corrected outputs. Measurements should be generated by running the code on the machine used for the assessment; do not present estimated runtimes as measured results.

Scenario: SentinelOps SOC Alert-Triage Module

SentinelOps is the Security Operations Centre of a cloud-services company.

(1) order alerts by severity, highest first, retaining arrival order for equal severities;

(2) determine whether an IP address is blocked;

(3) return every starting index of an attack signature in a log;

(4) identify the two distinct login events with the smallest Euclidean distance. Production workload is approximately 50,000 alerts and batches of up to 8,000 login events per day.

1. Original Code Under Investigation

```
import math

def sort_alerts(alerts):  # insertion sort, highest severity first
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j > 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j] = key
    return alerts

def is_blocked(blocklist, ip):  # sequential search
    for i in range(len(blocklist)):
        if blocklist[i] == ip:
            return False
        else:
            return False

def find_signature(log, pattern):  # all occurrences
    n, m = len(log), len(pattern)
    hits = []
    for i in range(n - m):
        for j in range(m):
            if log[i + j] != pattern[j]:
```

```
            break
        if j == m - 1:
            hits.append(i)
    return hits


def closest_logins(events):  # brute-force closest pair
    min_dist = 0
    pair = None
    for i in range(len(events)):
        for j in range(len(events)):
            d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
                          (events[i][1] - events[j][1]) ** 2)
            if d > min_dist:
                min_dist = d
                pair = (i, j)
    return pair, min_dist
```

2. Test Data and Expected Behaviour

```
alerts = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
queries = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
log, pattern = "FAILFAIXFAIL", "FAIL"
events = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]  # x, y in km
```

Function	Correct expected result	Reason
sort_alerts	[('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)]	Descending severity; A2 remains before A4 because their severities tie and A2 arrived first.
is_blocked	10.0.0.9 → True; 10.0.0.5 → True; 8.8.8.8 → False	Membership must be checked across the complete blocklist.
find_signature	[0, 8]	"FAIL" begins at character positions 0 and 8.
closest_logins	indices (0, 5), distance $\sqrt{2} \approx 1.414$ km	Events (2,3) and (3,4) are the nearest distinct locations.

(a) Reproduce and Identify the Defects

The supplied source has four functions. Several faulty statements produce more than one observable symptom. The task states that there are nine defects; the table below lists nine distinct logic defects or missing cases in the supplied code. A "defect" is counted as a specific incorrect condition, return value, or missing boundary check.

No.	Function / faulty expression	Defect and consequence
1	sort_alerts: `j > 0`	The loop excludes index 0. If the smallest existing element is at the first position, it cannot be shifted, so the insertion position can be wrong. Use `j >= 0`.
2	sort_alerts: `alerts[j]` after loop	The insertion assignment is only correct when the loop condition allows j to move to -1. Combined with defect 1, the first element is not handled correctly.
3	is_blocked: `return False` on match	A matching blocked IP is reported as not blocked. Return True when a match is found.
4	is_blocked: `else: return False` inside loop	The search stops after the first nonmatching element, so later matching entries are never examined. Continue the loop instead.
5	is_blocked: no explicit final result	A correct sequential search needs False after the entire list has been checked; an empty list should also return False.
6	find_signature: `range(n - m)`	Python's range excludes its endpoint; the last valid alignment at index n−m is never checked. Use `range(n - m + 1)`.
7	find_signature: `if j == m - 1`	After a mismatch, `break` can leave j at a value that makes the match test misleading; use an explicit full-match flag or `else` on the inner loop.
8	closest_logins: `min_dist = 0` and `if d > min_dist`	The algorithm selects a maximum non-zero distance, not the minimum distance. Initialise with infinity and update when d is smaller.
9	closest_logins: all i,j pairs including i == j	Self-pairs and both orders of the same pair are considered. Self-distance is zero and duplicate work wastes time; iterate only over `j > i` and handle fewer than two events.

Additional implementation concern: the original pattern-matching code does not safely handle an empty pattern because the inner loop variable `j` would never be assigned. The corrected version below defines an explicit policy: an empty pattern returns every boundary position from 0 through len(log). This makes the behaviour predictable and testable.

Original output assessment

For the supplied alert list, the original insertion sort can place the final low-severity alert incorrectly because the loop never shifts an element at index 0. For blocklist queries, the function returns False for every query because both branches return False and the function exits during the first iteration. For the log, `range(n-m)` excludes the last legal starting index; on the supplied input, it finds the occurrence at 0 but misses the one at 8. For closest logins, the comparison uses `>` and therefore searches for the farthest pair; it also compares each event with itself and visits each unordered pair twice.

Note on reproduction: run the original source exactly as provided to capture the literal console output. If the pasted code is missing indentation because of document formatting, restore Python indentation before execution. The results above describe the logic of the supplied function bodies.

(b) Explain Root Causes and Debug Systematically

B1. Insertion-sort trace

Insertion sort maintains a sorted prefix. For each next alert, it stores the alert in `key`, shifts lower-severity alerts one position right, and inserts the key in the resulting gap. The required order is descending by severity. The comparison must use `>= 0` so the algorithm can compare the key with the first item.

Pass	Key	Correct action	List after insertion
Start	—	Initial list	A1(3), A2(9), A3(5), A4(9), A5(1)
i = 1	A2(9)	Shift A1(3) right; insert A2 at index 0	A2(9), A1(3), A3(5), A4(9), A5(1)
i = 2	A3(5)	Shift A1(3); insert before A1	A2(9), A3(5), A1(3), A4(9), A5(1)
i = 3	A4(9)	Do not shift A2(9); stable tie order is retained	A2(9), A4(9), A3(5), A1(3), A5(1)
i = 4	A5(1)	No shift required	A2(9), A4(9), A3(5), A1(3), A5(1)

The trace above illustrates the intended invariant. In the actual corrected implementation, each pass operates on the list produced by the previous pass; equal severity values are not shifted because the condition uses a strict less-than comparison. That strict comparison is what preserves stability.

B2. Sequential blocklist search

A sequential search compares the requested IP against entries from left to right. On equality, it returns True immediately. On inequality, it continues to the next entry. If the loop finishes without a match, it returns False. The original code returns False on the first comparison regardless of whether it matches, so the second and third entries may never be reached.

Query	Trace	Correct result
10.0.0.9	Compare with 10.0.0.5 (no); compare with 10.0.0.9 (yes)	True
10.0.0.5	First comparison matches	True
8.8.8.8	Compare with all three entries; none match	False
Any IP, empty list	No iterations; return after loop	False

B3. Brute-force signature matching

For a log of length n and pattern length m, a valid starting index ranges from 0 through n−m inclusive. Thus, there are n−m+1 possible alignments when m ≤ n. The original loop uses `range(n-m)`, which omits the final alignment. The corrected implementation also needs an explicit success test so a partial match is not reported as a complete signature.

Start index	**Log slice**	**Comparison result**
0	FAIL	Full match — record 0
1	AILF	Mismatch
2	ILFA	Mismatch
3	LFAI	Mismatch
4	FAIX	Mismatch at final character
5	AIXF	Mismatch
6	IXFA	Mismatch
7	XFAI	Mismatch
8	FAIL	Full match — record 8

The log has 12 characters and the pattern has 4 characters, so valid starts are 0 through 8. The expected result is `[0, 8]`. If the pattern is longer than the log, there are no valid alignments and the result is an empty list. If the pattern is empty, this report's implementation returns all boundaries, which is a clearly documented convention.\

B4. Closest-login pair

The Euclidean distance between points (x_1,y_1) and (x_2,y_2) is $\sqrt{((x_1-x_2)^2+(y_1-y_2)^2)}$. To find the closest pair, the algorithm must minimise this value, not maximise it. The original code starts at zero and updates when the distance is greater, so it records the farthest pair it encounters. It also examines self-pairs and duplicate pair orders.

Pair indices	**Points**	**Squared distance**	**Distance**
(0, 5)	(2,3) and (3,4)	$1^2 + 1^2 = 2$	$\sqrt{2} \approx 1.414$ km
(0, 3)	(2,3) and (5,1)	$3^2 + (-2)^2 = 13$	$\sqrt{13} \approx 3.606$ km
(1, 4)	(12,30) and (12,10)	$0^2 + 20^2 = 400$	20 km

The closest pair in the supplied data is indices (0,5), with distance $\sqrt{2}$ km. With fewer than two events, no pair exists; the corrected function returns `(None, math.inf)` to signal that a distance could not be calculated. An alternative API could return `(None, None)`, but the selected convention must be documented and tested consistently.

B5. Retesting after each correction

A systematic debugging process changes one function at a time, runs its focused tests, and only then proceeds to the next function. This isolates causes and prevents a later change from masking an earlier defect.

1. Correct sort_alerts; test the sample, empty list, one item, already sorted list, reverse-sorted list, and equal severities.
2. Correct is_blocked; test first, middle, and absent values plus an empty blocklist.
3. Correct find_signature; test the sample, overlapping matches, final-position match, empty pattern, and pattern longer than the log.

4. Correct closest_logins; test the sample, duplicate points, exactly two points, and zero or one event.
5. Run the complete automated test harness after all functions are integrated.

(c) Optimise the Corrected Module

C1. Alert sorting

Insertion sort has $O(n^2)$ worst-case time and $O(n)$ best-case time on an already sorted list, with $O(1)$ auxiliary space. It is stable and performs well for small or nearly sorted inputs because it shifts only the out-of-order elements. For around 50,000 alerts, Python's built-in stable `sorted`/`list.sort` is generally preferable: Timsort is designed to exploit existing ordered runs and has $O(n \log n)$ worst-case time. If the assignment specifically requires the brute-force/insertion-sort technique, retain insertion sort for demonstrating the algorithm and benchmark it separately; for production, use the built-in stable sort.

C2. Repeated blocklist lookup

A sequential search costs $O(b)$ comparisons per query for a blocklist of size b. Converting the blocklist to a set costs $O(b)$ expected time and memory once, then each membership test is $O(1)$ on average. This is a strong choice when many alert IPs are checked against the same blocklist. Worst-case hash-table behaviour can degrade, but average-case performance is substantially better for ordinary use.

C3. Signature scanning

The corrected brute-force matcher checks each valid alignment exactly once and stops comparing an alignment as soon as a mismatch is found. Its worst-case time is $O((n-m+1)m)$, commonly stated as $O(nm)$, and its extra space is $O(k)$ for k recorded matches. For a single short signature this may be sufficient. For many signatures or very long logs, algorithms such as Knuth–Morris–Pratt can reduce matching time to $O(n+m)$ for one pattern, but the requested brute-force technique is retained here.

C4. Closest-pair search

The simple brute-force solution evaluates n(n−1)/2 distinct pairs. It should use squared distance, dx*dx + dy*dy, during comparisons and take one square root only for the final answer. Iterating with `j = i + 1` removes self-pairs and duplicate pair orders. This reduces unnecessary work but remains $O(n^2)$. A true asymptotic improvement for arbitrary 2D points is the divide-and-conquer closest-pair algorithm, which runs in O(n log n) under the standard geometric assumptions. The final module below uses the exact brute-force pair scan with the safe optimisations requested; it does not claim a quadratic scan has become subquadratic.

Optimisation summary

Component	**Baseline**	**Optimised choice**	**Complexity impact**
Sort alerts	Insertion sort	Stable built-in sort for production; insertion sort retained for comparison tests	$O(n^2)$ worst case to O(n log n) worst case
Blocklist	Sequential scan each query	Set membership	O(b) per query to O(1) average after O(b) setup
Signature	Brute force with bad bounds	Correct bounds and stop on mismatch	Same O(nm) worst case; avoids invalid/missed alignments
Closest pair	All ordered pairs and square roots	Unique pairs and squared distances	Still $O(n^2)$, about half as many pairs and fewer square roots

(d) Analyse Before and After

D1. Time and space complexity

Function / version	Basic operations	Time complexity	Auxiliary space
Insertion sort — original/corrected	Severity comparisons and shifts; worst-case shifts ≈ $n(n-1)/2$	Best $O(n)$; average/worst $O(n^2)$	$O(1)$ extra
Built-in stable sort — production	Comparison work depends on input runs	Worst $O(n \log n)$; near-sorted data can be faster	Implementation-dependent, typically $O(n)$
Sequential blocklist search	Up to b equality tests per query	$O(b)$ per query	$O(1)$
Set blocklist	Build b entries, then hash membership per query	$O(b)$ setup; expected $O(1)$ per query	$O(b)$
Brute-force signature	At most m character comparisons per alignment; up to $n-m+1$ alignments	$O(nm)$ worst case	$O(k)$ for k hits
Closest pair — original	n^2 distance calculations including self-pairs and duplicates	$O(n^2)$	$O(1)$
Closest pair — unique pairs	$n(n-1)/2$ squared-distance calculations	$O(n^2)$	$O(1)$
Closest pair — divide and conquer	Sort and recursively process point sets and strip	$O(n \log n)$	Typically $O(n)$ auxiliary space

D2. Closest-pair distance-computation counts

For n events, the original nested loops perform n^2 distance calculations. The unique-pair scan performs $n(n-1)/2$ squared-distance calculations. These are theoretical counts, not wall-clock measurements; the test harness in the final code prints the actual counters for n = 1,000, 2,000, 4,000 and 8,000. The square-root count in the optimised version is one per successful final result, rather than one per candidate pair.

n events	**Original n^2 calculations**	**Unique pairs $n(n-1)/2$**	**Reduction in pair calculations**
1,000	1,000,000	499,500	50.05%
2,000	4,000,000	1,999,000	50.03%
4,000	16,000,000	7,998,000	50.01%
8,000	64,000,000	31,996,000	50.01%

Interpretation: removing self-pairs and repeated pair orders cuts the candidate-pair count by approximately half. It does not change the $O(n^2)$ growth rate. At n = 8,000, the unique-pair scan still evaluates nearly 32 million pairs, so a divide-and-conquer algorithm is the appropriate next step if processing time becomes unacceptable.

D3. Insertion-sort comparison experiment

For a reproducible experiment, generate the same number of alerts in two arrangements: (1) already sorted or nearly sorted by descending severity, and (2) randomly shuffled. Count each severity comparison inside the insertion-sort while condition. The exact counts depend on the generated data and the number of displaced elements, so they should be reported from a real run rather than invented.

Input arrangement	Expected comparison behaviour	Explanation
Already sorted descending	Approximately n−1 key comparisons	Each new key is already in the correct position; no shifts are required.
Nearly sorted descending	Close to linear if only a small number of elements are displaced	Insertion sort shifts only elements that cross an out-of-order item.
Random order	Typically quadratic growth	Many keys must move across a substantial fraction of the sorted prefix.
Reverse order for descending target	Quadratic worst case	Each new item must be shifted to the front of the current prefix.

Measurements should include list size, arrangement, comparison count, and elapsed time. Run each case several times and report the median runtime. Runtime varies with hardware, interpreter version, and background load; comparison counts are more reproducible than elapsed time.

D4. Reproducible benchmark snippet

```
# Add this to the test script or run it separately.
import random
import time


def insertion_sort_count(alerts):
    data = list(alerts)
    comparisons = 0
    for i in range(1, len(data)):
        key = data[i]
```

```
        j = i - 1
        while j >= 0:
            comparisons += 1
            if data[j][1] < key[1]:
                data[j + 1] = data[j]
                j -= 1
            else:
                break
        data[j + 1] = key
    return data, comparisons


rng = random.Random(42)
for n in (1000, 5000, 10000):
    base = [(f"A{i}", rng.randrange(1, 11)) for i in range(n)]
    nearly_sorted = sorted(base, key=lambda item: item[1], reverse=True)
    random_order = list(base)
    rng.shuffle(random_order)
    for label, sample in (("nearly sorted", nearly_sorted),
                          ("random", random_order)):
        start = time.perf_counter()
        _, comparisons = insertion_sort_count(sample)
        elapsed = time.perf_counter() - start
        print(n, label, comparisons, f"{elapsed:.6f}s")
```

(e) Deliver Production-Quality Code

The implementation below includes docstrings, stable ordering, explicit boundary behaviour, counters for algorithm analysis, and assertions covering the required cases. The closest-pair implementation is a brute-force scan with unique pairs and squared-

distance comparisons, suitable for demonstrating the optimisation requested in the assessment. For an actual high-throughput production deployment, replace it with a tested O(n log n) divide-and-conquer implementation if benchmark results justify the additional complexity.

Final optimised Python module

```
"""SentinelOps alert-triage algorithms.

Coordinates are represented as (x, y) tuples in kilometres.
Alerts are (alert_id, severity) tuples and are sorted stably by severity.
"""

import math
from typing import Iterable, Optional



def insertion_sort_alerts(alerts):
    """Return a stable descending-severity copy using insertion sort.

    Kept for algorithm analysis and nearly-sorted input experiments.
    Equal-severity alerts retain their original arrival order.
    """
    result = list(alerts)
    for i in range(1, len(result)):
        key = result[i]
        j = i - 1
        while j >= 0 and result[j][1] < key[1]:
            result[j + 1] = result[j]
            j -= 1
```

```
        result[j + 1] = key
    return result


def sort_alerts(alerts):
    """Return alerts sorted by severity descending, stably.

    Python's built-in sort is stable and is preferred for large production
    batches. The input list is not modified.
    """
    return sorted(alerts, key=lambda alert: alert[1], reverse=True)


def is_blocked(blocklist, ip):
    """Return True if ip is in blocklist, otherwise False."""
    return ip in blocklist


def make_blocklist_index(blocklist):
    """Build a set for efficient repeated membership checks."""
    return set(blocklist)


def find_signature(log, pattern):
    """Return all start indices where pattern occurs in log.

    Overlapping matches are included. By convention, an empty pattern
    matches every boundary position, from 0 through len(log).
    """
```

```
    n, m = len(log), len(pattern)
    if m == 0:
        return list(range(n + 1))
    if m > n:
        return []

    hits = []
    for i in range(n - m + 1):
        for j in range(m):
            if log[i + j] != pattern[j]:
                break
        else:
            hits.append(i)
    return hits


def closest_logins(events):
    """Return ((i, j), distance) for the closest distinct pair.

    Each unordered pair is examined once. Squared distances are compared
    to avoid square roots inside the loop. If fewer than two events exist,
    return (None, math.inf).
    """
    n = len(events)
    if n < 2:
        return None, math.inf

    best_pair = None
    best_sq = math.inf
```

```
    for i in range(n - 1):
        x1, y1 = events[i]
        for j in range(i + 1, n):
            x2, y2 = events[j]
            dx = x1 - x2
            dy = y1 - y2
            dist_sq = dx * dx + dy * dy
            if dist_sq < best_sq:
                best_sq = dist_sq
                best_pair = (i, j)

    return best_pair, math.sqrt(best_sq)



def closest_logins_with_count(events):
    """Return ((pair, distance), pair_comparisons) for benchmarking."""
    n = len(events)
    if n < 2:
        return (None, math.inf), 0

    best_pair = None
    best_sq = math.inf
    comparisons = 0
    for i in range(n - 1):
        x1, y1 = events[i]
        for j in range(i + 1, n):
            x2, y2 = events[j]
            dx = x1 - x2
            dy = y1 - y2
```

```
            dist_sq = dx * dx + dy * dy
            comparisons += 1
            if dist_sq < best_sq:
                best_sq = dist_sq
                best_pair = (i, j)
    return (best_pair, math.sqrt(best_sq)), comparisons


def brute_force_ordered_pair_count(n):
    """Return the number of ordered-pair distance evaluations in the old loop."""
    return n * n


def unique_pair_count(n):
    """Return the number of distinct unordered pairs."""
    return n * (n - 1) // 2


def run_tests():
    """Run correctness and boundary tests for all functions."""
    alerts = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
    expected = [("A2", 9), ("A4", 9), ("A3", 5), ("A1", 3), ("A5", 1)]
    assert sort_alerts(alerts) == expected
    assert insertion_sort_alerts(alerts) == expected
    assert sort_alerts([]) == []
    assert sort_alerts([("only", 4)]) == [("only", 4)]
    assert sort_alerts([("x", 5), ("y", 5)]) == [("x", 5), ("y", 5)]

    blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
```

```
    assert is_blocked(blocklist, "10.0.0.9") is True
    assert is_blocked(blocklist, "8.8.8.8") is False
    assert is_blocked([], "8.8.8.8") is False
    index = make_blocklist_index(blocklist)
    assert is_blocked(index, "10.0.0.5") is True

    assert find_signature("FAILFAIXFAIL", "FAIL") == [0, 8]
    assert find_signature("AAAA", "AA") == [0, 1, 2]
    assert find_signature("ABC", "BC") == [1]  # pattern at final valid start
    assert find_signature("AB", "LONG") == []
    assert find_signature("ABC", "") == [0, 1, 2, 3]
    assert find_signature("", "A") == []

    events = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]
    pair, distance = closest_logins(events)
    assert pair == (0, 5)
    assert math.isclose(distance, math.sqrt(2))
    assert closest_logins([]) == (None, math.inf)
    assert closest_logins([(1, 1)]) == (None, math.inf)
    assert closest_logins([(0, 0), (0, 0)]) == ((0, 1), 0.0)

    _, count = closest_logins_with_count(events)
    assert count == unique_pair_count(len(events))
    assert brute_force_ordered_pair_count(1000) == 1_000_000
    assert unique_pair_count(1000) == 499_500
    print("All correctness and boundary tests passed.")



def benchmark_closest_pair():
```

```
    """Print theoretical and measured pair counts for 1k–8k events."""
    import random
    import time

    rng = random.Random(42)
    for n in (1000, 2000, 4000, 8000):
        points = [(rng.random() * 1000, rng.random() * 1000)
                  for _ in range(n)]
        start = time.perf_counter()
        _, actual_count = closest_logins_with_count(points)
        elapsed = time.perf_counter() - start
        print(
            f"n={n:5d} | pairs={brute_force_ordered_pair_count(n):>10d} "
            f"| unique pairs={actual_count:>10d} | elapsed={elapsed:.4f}s"
        )


if __name__ == "__main__":
    run_tests()
    # Uncomment the following line to run the potentially slower benchmark.
    # benchmark_closest_pair()
```

How to Run and Verify

6. Save the final code in a file named `sentinelops_algorithms.py`.
7. Run `python sentinelops_algorithms.py` using Python 3. The expected final line is: `All correctness and boundary tests passed.`
8. To benchmark closest-pair operations, uncomment `benchmark_closest_pair()` under the main guard and run the file. Record the printed counts and runtimes from your own computer.

9. Use `insertion_sort_count` from the benchmark snippet to compare nearly sorted and random alerts. Record the actual output rather than copying example timings.
10. Commit the final source and report to your GitHub repository, then submit the repository URL through Google Classroom as required by the assessment.

Conclusion

The defects arose primarily from incorrect loop boundaries, early returns, inverted comparison logic, and unnecessary pair enumeration. Correcting the conditions restores the intended behaviour; boundary tests make the fixes repeatable. The strongest production improvements are stable built-in sorting for large batches and set-based IP membership. Brute-force signature matching is correct once its alignment range and match test are fixed. The closest-pair scan can be made more efficient by checking each unordered pair once and comparing squared distances, but its $O(n^2)$ growth remains; a divide-and-conquer method is recommended if the 8,000-event benchmark is too slow.